SACOR

SOLUCIONES AMBIENTALES CORPORATIVAS

SACOR SYSTEM

Technical Manual

Installation, configuration and maintenance of the web, mobile, desktop and WhatsApp system

PARTS 1 · 2

Installation

Architecture, requirements and setup of the applications and the chatbot

PART 3

Security

Application keys, employee access and production

PARTS 4 · 5

Maintenance

Database, backups, updates and troubleshooting

PRESENTED BY

William Ariel Mejía Mira **José Steven Menéndez Benítez** **Diego Josué Elías López**

Colegio Español Padre Arrupe

Technical Vocational High School Diploma in Software Development

Version 1.1

October 2026

How to use this manual

This manual explains how to install, configure and maintain SACOR System. It is written for the person in charge of the server, not for the daily users of the applications. It is divided into five parts, and the **colored tab** on the right edge takes you quickly to each one.

Part 1 · Architecture OVERVIEW
Understand the components of SACOR System, how they connect and what the server and the client devices need. p. 4

Part 2 · Installation SETUP
Install the database, the API, the website, the desktop app, the mobile app and the WhatsApp chatbot in the right order. p. 9

Part 3 · Security ADMINISTRATION
Manage application keys, employee access and mobile sessions, and prepare the system for production. p. 20

Part 4 · Database and Maintenance OPERATION
Know the main tables, make backups, run useful queries and install updates safely. p. 26

Part 5 · Troubleshooting SUPPORT
Find the cause and the solution of the most common problems of the installation. p. 31

How to read each page

☆ Section — Each topic is presented as a **section** with its steps in order.

1 — **Numbered steps** must be followed in the order shown.

```
git pull
```
Dark windows contain commands or file contents to copy exactly.

Tip: a recommendation to make the task easier.

Caution!: something that can cause errors or data loss.

Note: additional information about the topic.

Before you start

Administrator rights

You need an administrator account on the Windows server to install programs and open ports.

Basic console use

The steps use PowerShell, SQL Server Management Studio and Git commands.

Source code

Access to the four GitHub repositories of the project: API, website, desktop and mobile.

Contents

Part 1 · Architecture — OVERVIEW

1	System architecture	5
2	How the applications identify themselves	6
3	Software requirements	7
4	Server hardware and client devices	8

Part 2 · Installation — SETUP

1	Installation order	10
2	Create the database	11
3	Configure and start the API	12
4	Check the API and open the network	13
5	Install the website	14
6	Install the desktop app	15
7	Run the mobile app	16
8	Build the mobile installer (APK)	17
9	Set up the WhatsApp chatbot	18
10	Chatbot conversation	19

Part 3 · Security — ADMINISTRATION

1	Application keys	21
2	Employee access and mobile sessions	22
3	Protections of the API	23
4	Production deployment	24
5	Security checklist	25

Part 4 · Database and Maintenance — OPERATION

1	Main tables	27
2	Backups and restore	28
3	Useful queries and stored procedures	29
4	Updating the system	30

Part 5 · Troubleshooting — SUPPORT

1	Common problems	32
2	More problems	33

PART 1

Architecture

Overview

Understand the components of SACOR System, how they connect and what the server and the client devices need.

In this part you will learn to:

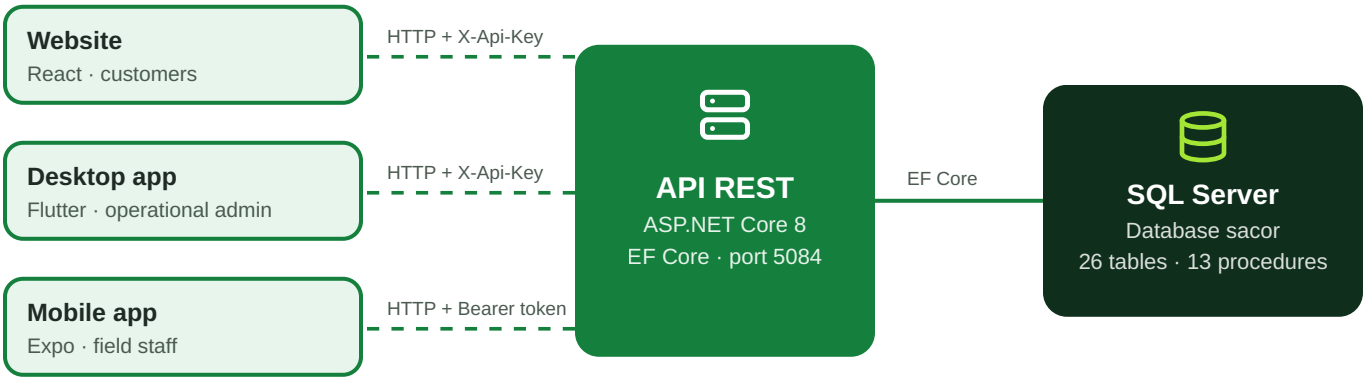
1. Understand the architecture of the system
2. Know how each application identifies itself
3. Check the software requirements
4. Prepare the server and the client devices



Section 1

System architecture

SACOR System has a central API that is the only component connected to the database. The three client applications always communicate with the API through HTTP requests in JSON format.



Component	Technology	Used by
Database	SQL Server 2022	API only
API	C# with ASP.NET Core 8 and Entity Framework Core	All the applications
Website	React 19 with Vite	Customers and job applicants
Desktop app	Flutter for Windows	Operational administrator
Mobile app	React Native with Expo	Technical administrator, technicians and drivers
WhatsApp chatbot	Twilio WhatsApp + webhook in the API	Customers

Note

The database never receives connections from the applications. If the API is stopped, none of the applications can read or save data.

Section 2

How the applications identify themselves

Every request that reaches the API must prove which application sent it. The mechanism is different for each client.

Website and desktop app

They send a secret key in the `X-Api-Key` header of every request. Each application has its own key, so one key can be changed without affecting the other application.

Mobile app

A key stored inside an installed application can be extracted, so the mobile app does not use a shared key. Each employee signs in with a personal access code and the API answers with an individual session token, valid for 30 days. Only a hash of the token is stored in the database.

User types

User type	Value in the database	Application
Operational administrator	AdministradorOperativo	Desktop app
Technical administrator	AdministradorTecnico	Mobile app
Technician	Tecnico	Mobile app
Driver	Motorista	Mobile app

Caution!

The former types Ayudante, Mecanico and Coordinador are kept only for historical records. The database and the API do not allow those employees to be active.

Section 3

Software requirements

Install these programs on the server or on the computer used to compile the applications before starting Part 2.

Software	Version	Purpose
Windows	10 or 11, 64 bits	Operating system of the server and of the desktop app
SQL Server and SSMS	2022 (Express or Developer)	Database
.NET SDK	8.0	Run and compile the API
Node.js	20 or later	Website and mobile app
Flutter SDK and Visual Studio 2022	Flutter 3.47 or later, with Desktop development with C++	Compile the desktop app
Android Studio or Expo Go	Current version	Run the mobile app
Git	Current version	Download and update the source code

Tip

Install SQL Server first and write down the instance name shown at the end of the installation, for example **localhost\SQLEXPRESS**. It is needed in the connection string of the API.

Section 4

Server hardware and client devices

The server runs SQL Server and the API. It can be the computer of the operational administrator if it stays on during working hours.

Server

Resource	Minimum	Recommended
Operating system	Windows 10, 64 bits	Windows 11 or Windows Server
Processor	4 cores	6 cores or more
RAM	8 GB	16 GB
Disk	20 GB free	SSD with 50 GB free
Network	Local network, cable or Wi-Fi	Fixed IP address in the local network

Client devices

User	Application	Device
Operational administrator	Desktop app	Windows 10 or 11, 64 bits, with the Microsoft Visual C++ Redistributable (x64)
Technical administrator, technician and driver	Mobile app	Android phone with Android 7.0 or later
Customers	Website	Any device with an updated browser

Note

Phones and computers must reach the server through the local network, or the API must be published on a public address with HTTPS (Part 3, Section 4).

PART 2

Installation

Setup

Install the database, the API, the website, the desktop app, the mobile app and the WhatsApp chatbot in the right order.

In this part you will learn to:

1. Follow the installation order
2. Create the database
3. Configure and start the API
4. Check the API and open the network
5. Install the website
6. Install the desktop app
7. Run the mobile app
8. Build the mobile installer (APK)
9. Set up the WhatsApp chatbot
10. Know the chatbot conversation

Section 1

Installation order

Each application depends on the API, and the API depends on the database. Install the components in this order.

Step	What is installed	Section
1	Required software: SQL Server, .NET 8 SDK, Node.js, Flutter and Git	Part 1 · 3
2	Database sacor	Part 2 · 2
3	API with its keys	Part 2 · 3
4	Connection checks and network access	Part 2 · 4
5	Website	Part 2 · 5
6	Desktop app of the operational administrator	Part 2 · 6
7	Mobile app and its installer	Part 2 · 7 and 8
8	WhatsApp chatbot	Part 2 · 9 and 10
9	Production settings	Part 3 · 4 and 5

Tip

Keep a text file with the server name, the IP address and the location of each key while you install. You will need them in several steps.

Production

Part 3, Sections 4 and 5, are applied only when the system is used with real customers and employees.

Section 2

Create the database

The file `db/sacor_completo.sql` of the API repository creates the complete database in a single run.

1. Open SQL Server Management Studio and connect to the instance.
2. Open the file **db/sacor_completo.sql** with **File › Open › File**.
3. Press **F5** to run the complete file.
4. Refresh the Object Explorer: the database **sacor** appears with 26 tables and 13 stored procedures, one truck and one employee of each type.

> **Caution!**
> The script deletes the tables if the database already exists. Run it only on a new installation or when the data can be lost. Make a backup first (Part 4, Section 2).

Initial users

The script creates these users so the system can be used immediately:

User type	Access code	Application
Operational administrator	710245	Desktop app
Technical administrator	820376	Mobile app
Technician	3104	Mobile app
Driver	4215	Mobile app

> **Tip**
> Change these users to **Inactivo** after the real employees are registered.

Section 3

Configure and start the API

The API is a .NET 8 project. Its secrets are kept outside the source code with user-secrets.

1 Download the repository and open a terminal in the project folder:

POWERSHELL

```
git clone https://github.com/RuruiYo/TesisSoftware2026.git
cd TesisSoftware2026\APISacor
```

2 Write the connection string in **appsettings.Development.json**. This file is not uploaded to Git:

APPSETTINGS.DEVELOPMENT.JSON

```
"ConnectionStrings": {
  "SacorDb":
"Server=SERVER_NAME;Database=sacor;Trusted_Connection=True;TrustServerCertificate=True;Encrypt=Tru
}
```

3 Generate a random key for the desktop app and save it with user-secrets. Repeat with **Seguridad:ApiKeys:AppWeb** for the website:

POWERSHELL · FOLDER APISacor

```
$bytes = New-Object byte[] 32
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($bytes)
$llave = [Convert]::ToBase64String($bytes)
dotnet user-secrets set "Seguridad:ApiKeys:AppEscritorio" $llave
$llave
```

4 Start the API and keep the terminal open:

POWERSHELL

```
dotnet run --launch-profile http
```

Section 4

Check the API and open the network

Confirm that the API answers before installing the client applications.

Address	Expected result
http://localhost:5084/health	{"estado":"ok"}
http://localhost:5084/swagger	Interactive documentation of the endpoints (Development only)

The **http** profile listens only on localhost, which is enough for the Android emulator (it reaches the computer through **10.0.2.2**). To accept real phones and other computers, start the API on all the network cards and open the port in the firewall (console as administrator):

POWERSHELL · ADMINISTRATOR

```
dotnet run --launch-profile http -- --urls "http://0.0.0.0:5084"
netsh advfirewall firewall add rule name="SACOR API" dir=in action=allow protocol=TCP
localport=5084
ipconfig
```

1. Write down the **IPv4 address** shown by ipconfig, for example 192.168.1.20.
2. Use **http://192.168.1.20:5084** as the address of the API in the other devices.
3. From a phone, open **http://192.168.1.20:5084/health** in the browser to test the connection.

Tip
Reserve the IP address of the server in the router so it does not change.

Caution!
On a phone, never use localhost: that address points to the phone itself.

Section 5

Install the website

The public website is a React project built with Vite.

POWERSHELL

```
git clone https://github.com/RuruiYo/SACORWEB.git
cd SACORWEB
npm install
copy .env.example .env.local
```

Complete the file **.env.local** with these values:

Variable	Value
VITE_API_URL	Address of the API, for example http://localhost:5084
VITE_API_KEY	The AppWeb key generated in Section 3
VITE_WHATSAPP_SACOR	WhatsApp number of SACOR with the country code, without spaces

1. Run `npm run dev` to test the website on http://localhost:5173.
2. Run `npm run build` to generate the files that are published on a web server.

Caution!

The address of the website must be in the list **Seguridad:OrigenesPermitidos** of the API; otherwise the browser blocks the requests.

Section 6

Install the desktop app

The desktop app of the operational administrator is a Flutter project for Windows.

POWERSHELL

```
git clone https://github.com/RuruiYo/SACOR-DESKTOP.git
cd SACOR-DESKTOP
flutter pub get
```

1. Create the file **sacor_config.json** in the project folder, or next to SACOR.Desktop.exe in a compiled installation:

SACOR_CONFIG.JSON

```
{
  "apiUrl": "http://localhost:5084",
  "apiKey": "AppEscritorio key generated in Section 3",
  "modoDemo": false
}
```

2. Run `flutter run -d windows` to test it.
3. To deliver it to other computers, run `herramientas\compilar_windows.ps1 -ApiKey "key"`. It creates **build\SACOR.Desktop.zip**, which is extracted on each computer.

> **Note**
> The desktop app accepts only active operational administrators, closes the session after 30 minutes without use and blocks the sign-in for one minute after 5 failed attempts.

Section 7

Run the mobile app

The mobile app is an Expo project for the technical administrator, technicians and drivers.

POWERSHELL

```
git clone https://github.com/wa2m-cs/AppMovilTesis.git
cd AppMovilTesis
npm install
```

1. Create the file **.env.local** with the address of the API:

.ENV.LOCAL

```
Android emulator
EXPO_PUBLIC_API_URL=http://10.0.2.2:5084
Real phone in the local network
EXPO_PUBLIC_API_URL=http://192.168.1.20:5084
```

2. Run `npx expo start -c`.
3. Press **a** to open the app in the emulator, or scan the QR code with Expo Go on a phone connected to the same network.
4. Sign in with one of the access codes of Section 2.

Tip

Use `-c` every time you change .env.local: it clears the cache so the new address is used.

Section 8

Build the mobile installer (APK)

Expo Go is only for testing. To install the app on the phones of the employees, generate an APK with EAS, the build service of Expo (free account).

1. Write the final address of the API in **.env.local**. It is copied inside the APK, so generate the APK again if the server address changes.
2. In **app.json**, add the Android package inside "expo": `"android": { "package": "com.sacor.app" }`
3. Sign in to Expo and create the build configuration (creates eas.json):

POWERSHELL

```
npx eas-cli login
npx eas-cli build:configure
```

4. In **eas.json**, make the preview profile generate an APK:

EAS.JSON

```
"preview": { "distribution": "internal", "android": { "buildType": "apk" } }
```

5. Compile with `npx eas-cli build --platform android --profile preview`.
6. Download the APK from the link that EAS shows, copy it to each phone, allow apps from unknown sources and install it.

Caution!

Android blocks HTTP in installed apps. While the API has no HTTPS, run `npx expo install expo-build-properties` and add `["expo-build-properties", { "android": { "usesCleartextTraffic": true } }]` to the plugins of app.json.

Section 9

Set up the WhatsApp chatbot

Customers can request a quote by writing to the SACOR WhatsApp number. Twilio receives each message and sends it to the API, which answers with the next question.

FLOW OF A MESSAGE

```
Customer (WhatsApp)  ->  Twilio  ->  https://TUNNEL/api/whatsapp/webhook
API: checks X-Twilio-Signature, saves the step in conversacion_whatsapp
API  ->  Twilio  ->  Customer: next question (TwiML)
Last step: usuario_web + cotizacion (Pendiente, Origen: WhatsApp)
```

1. Create an account at **twilio.com** and open **Messaging › Try it out › Send a WhatsApp message**.
2. From the phone, send to the sandbox number the message **join <keyword>** shown by Twilio.
3. Start the API with the http profile and open a public tunnel:

POWERSHELL

```
winget install --id Cloudflare.cloudflared
cloudflared tunnel --url http://localhost:5084
```

4. In **Sandbox settings › When a message comes in**, write `https://TUNNEL/api/whatsapp/webhook` with method POST and save.
5. Save the Auth Token (Account Dashboard › Account Info) and the same webhook address:

POWERSHELL · FOLDER APISacor

```
dotnet user-secrets set "Twilio:AuthToken" "AUTH_TOKEN"
dotnet user-secrets set "Twilio:UrlWebhook" "https://TUNNEL/api/whatsapp/webhook"
```

Caution!

The address saved in Twilio:UrlWebhook must be identical to the one configured in Twilio, because it is part of the signature. The quick tunnel changes its address every time it starts: update both places when it changes.

Section 10

Chatbot conversation

The state of each conversation is saved in the table conversacion_whatsapp, so the customer can answer at their own pace.

Step	Question	Validation
menu	1. Request a quote · 2. Talk to an advisor	Option 2 silences the bot for 12 hours or until the customer writes menu
nombre	Full name or company	3 to 150 characters
dui	DUI	Format 00000000-0
correo	Email	Valid email or omitir
servicio	Type of service (1 to 4)	Same services as the website
modalidad	One-time service or recurring contract	1 or 2
lugar / destino	Collection place and destination	Destination only for hauling and specific services
materiales	Materials by number	Types registered in precio_lugar
duracion, frecuencia	Months and frequency (recurring only)	1 to 36 months
fecha	Date dd/mm/aaaa	From today up to two years
volumen, descripcion	Volume and details	Optional
confirmar	Summary: 1 send, 2 cancel	Creates the quote and replies with its number

Note

The customer can write **cancelar** at any moment. A conversation left unfinished for 2 hours starts again. The quote appears in the desktop app under Solicitudes web, with the detail **Origen: WhatsApp**.

Production

The sandbox only answers numbers that sent the join message. For real customers, register a WhatsApp sender with the company number in Twilio and use a fixed HTTPS address instead of the quick tunnel.

PART 3

Security

Administration

Manage application keys, employee access and mobile sessions, and prepare the system for production.

In this part you will learn to:

1. Rotate the application keys
2. Manage employee access codes and mobile sessions
3. Know the protections of the API
4. Prepare the API for production
5. Review the security checklist



Section 1

Application keys

The website and the desktop app authenticate with the keys AppWeb and AppEscritorio.

Change a key when it was shared by mistake, when a member of the team leaves the project, or at least once a year.

1. Generate the new key with the commands of Part 2, Section 3.
2. Save it with user-secrets (development) or as an environment variable (production).
3. Restart the API. The old key stops working immediately.
4. Update the key in **.env.local** of the website and publish it again, or in **sacor_config.json** of each desktop installation.

POWERSHELL · CHECK THE SAVED KEYS

```
dotnet user-secrets list
```

Caution!
Never write the keys in appsettings.json, in the source code or in a chat. The files .env.local and sacor_config.json are excluded from Git on purpose.

Section 2

Employee access and mobile sessions

Employees sign in with a personal access code that works as their password.

Access codes

The code is generated automatically when the employee is registered: **6 digits** for administrators and **4 digits** for technicians and drivers. Deliver it in private. The operational administrator can see it in the desktop app with the option **Ver código** (Show code).

To block an employee, change the state to **Inactivo** or **Suspendido**. The API rejects the sessions of inactive employees immediately, even if the session was open.

Close the mobile sessions of an employee

For example when a phone is lost. Replace the number with the id of the employee:

SQL SERVER MANAGEMENT STUDIO

```
UPDATE sesion_movil
SET revocado_utc = SYSUTCDATETIME()
WHERE id_empleado = 5 AND revocado_utc IS NULL;
```

Note

A mobile session lasts 30 days. Only a hash of the token is saved in the database, so a copy of the table does not allow anyone to sign in.

Section 3

Protections of the API

These protections are always active and do not need configuration, except the allowed origins.

Protection	Configuration
Request limit	120 requests per minute for each key; 5 sign-in attempts per minute for each address
Allowed origins	Only the addresses listed in Seguridad:OrigenesPermitidos can call the API from a browser
Size limits	1 MB for each request; 3 MB for each photo, with a maximum of 3 photos per evidence
Validation	Lists of allowed values for states, user types, expense concepts and evidence types; removal of HTML and control characters
Deletion	Deletions that would remove related records are rejected with the code 409 Conflict
Audit	Every change made from the desktop app and every assignment made from the mobile app is stored in the table auditoria
Error messages	The user receives a generic message; the details are written only in the console of the API
WhatsApp webhook	Only requests signed by Twilio with X-Twilio-Signature are accepted

Tip

If the website stops sending forms after changing its address, add the new address to Seguridad:OrigenesPermitidos and restart the API.

Section 4

Production deployment

Development mode is only for testing. With real data, the API runs in Production mode: Swagger is disabled and every request is redirected to HTTPS.

Environment variables

user-secrets only works in Development. In production the values are environment variables of the server; two underscores replace each colon. Run in a console as administrator:

POWERSHELL · ADMINISTRATOR

```
setx ASPNETCORE_ENVIRONMENT "Production" /M
setx ConnectionStrings__SacorDb
"Server=SERVER_NAME;Database=sacor;Trusted_Connection=True;TrustServerCertificate=True;Encrypt=Tru
M
setx Seguridad__ApiKeys__AppWeb "AppWeb key" /M
setx Seguridad__ApiKeys__AppEscritorio "AppEscritorio key" /M
setx Seguridad__OrigenesPermitidos__0 "https://address-of-the-website" /M
```

Publish the API

POWERSHELL

```
dotnet publish -c Release -o C:\SACOR\Api
cd C:\SACOR\Api
APISacor.exe --urls "https://0.0.0.0:7210;http://0.0.0.0:5084"
```

HTTPS needs a .pfx certificate, indicated with `Kestrel__Certificates__Default__Path` and `Kestrel__Certificates__Default__Password`. Then change the address to https in the website, the desktop app and the mobile app, and build the APK again.

Section 5

Security checklist

Review each point before the system is used with real data, and again after every update.

- [ ] HTTPS is configured with a valid certificate, and every application uses the https address of the API.
- [ ] ASPNETCORE_ENVIRONMENT is **Production**, so Swagger is not available.
- [ ] The connection string, the application keys and the Twilio Auth Token are environment variables; they are not in appsettings.json or in Git.
- [ ] Seguridad:OrigenesPermitidos contains only the real address of the website.
- [ ] The test employees created by sacor_completo.sql (710245, 820376, 3104 and 4215) are **Inactivo**.
- [ ] The ports of the API are open only to the networks that need them; SQL Server (port 1433) is never open to the internet.
- [ ] Daily backups are scheduled and copied to another disk (Part 4, Section 2).
- [ ] Only authorized personnel can sign in to the server and to SQL Server.

Caution!
The access codes are the passwords of the employees. Ask each employee not to share it, and deactivate the employees who leave the company on the same day.

PART 4

Database and Maintenance

Operation

Know the main tables, make backups, run useful queries and install updates safely.

In this part you will learn to:

1. Know the main tables
2. Make and restore backups
3. Run useful queries
4. Install updates of the system



Section 1

Main tables

The database sacor has 26 tables grouped by area. The entity-relationship diagram is in db/diagrama_er_chen_sacor.drawio.

Area	Tables
Personnel	empleado, pago, anticipo_sueldo, horario_trabajo, observacion_empleado
Customers and contracts	cliente, precio_lugar, servicio_contrato, cotizacion, usuario_web, solicitud_puesto_trabajo
Trips	ruta_trabajo, ruta_empleado, camion, camion_conductor, viaje_extra, viaje_extra_empleado
Field work	pesaje, limpieza, evidencia_foto, gasto_viaje
Security	sesion_movil, codigo_activacion_movil, auditoria
WhatsApp chatbot	conversacion_whatsapp

How the data is related

1. A contract (**servicio_contrato**) has several trips (**ruta_trabajo**).
2. Each trip has its own personnel (**ruta_empleado**) and truck.
3. Weighings, evidence and expenses are linked to the trip where they happened.
4. Each weighing stores the price per ton valid when it was registered, so a later change of price does not modify past sales.

Caution!

Do not edit sales, weighings or audit records directly in the tables. Use the applications so every change is validated and audited.

Section 2

Backups and restore

Make a backup every day and keep at least the last seven copies, on a disk different from the one of the server.

Make a backup

SQL SERVER MANAGEMENT STUDIO

```
BACKUP DATABASE sacor
TO DISK = 'D:\Respaldos\sacor.bak'
WITH INIT, COMPRESSION, NAME = 'Respaldo diario SACOR';
```

Restore a backup

Stop the API first, so no application is connected:

SQL SERVER MANAGEMENT STUDIO

```
USE master;
ALTER DATABASE sacor SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
RESTORE DATABASE sacor FROM DISK = 'D:\Respaldos\sacor.bak' WITH REPLACE;
ALTER DATABASE sacor SET MULTI_USER;
```

Frequency	Task
Daily	Backup of the database (Windows Task Scheduler with sqlcmd)
Weekly	Copy the backups to an external disk or to the cloud
Monthly	Check the free disk space of the server
Before each update	Make a manual backup

Section 3

Useful queries and stored procedures

Queries that help the system administrator check the activity of the system.

Latest changes in the audit

```sql
SELECT TOP 50 a.fecha_hora, e.nombre, a.tabla_afectada, a.accion, a.detalle
FROM auditoria a
LEFT JOIN empleado e ON e.id_empleado = a.id_empleado
ORDER BY a.fecha_hora DESC;
```

Active employees and their access codes

```sql
SELECT codigo_empleado, nombre, tipo
FROM empleado
WHERE estado = 'Activo'
ORDER BY tipo, nombre;
```

Stored procedures

The database includes 13 stored procedures for direct operations, for example `sp_RegistrarPesaje`, `sp_RegistrarGastoViaje` and `sp_MarcarEntradaEmpleado`. The API does not use them; they apply the same rules and are useful for corrections or massive loads.

Caution!
The result of the second query contains the access codes. Do not share it or save it in shared folders.

Section 4

Updating the system

Each component has its own Git repository. Install updates outside working hours.

1. Make a backup of the database (Section 2).
2. Download the changes in each folder with `git pull`.
3. **API:** run `dotnet build` and start the API again. If the update changes the database, the change is included in db/sacor_completo.sql.
4. **Website:** run `npm install` and `npm run build`, and publish the new files.
5. **Desktop app:** compile again with compilar_windows.ps1 and replace the files on each computer, keeping sacor_config.json.
6. **Mobile app:** run `npx expo install` and build a new APK if the app changed.
7. Review the security checklist (Part 3, Section 5).

POWERSHELL · EXAMPLE FOR THE API

```
cd C:\SACOR\TesisSoftware2026\APISacor
git pull origin main
dotnet build
```

Tip

If git push is rejected because a teammate uploaded changes, run `git pull --rebase origin main` and push again.

PART 5

Troubleshooting

Support

Find the cause and the solution of the most common problems of the installation.

In this part you will learn to:

1. Identify the cause of common errors
2. Apply the solution step by step
3. Know what to collect before asking for help



Section 1

Common problems

Quick solutions to the most common problems of the installation.

The desktop app shows «Falta la llave de la API».

Cause: sacor_config.json does not exist or apiKey is empty.

Solution: Create the file next to the executable with the AppEscritorio key.

The API answers 401.

Cause: The key sent by the application does not match the key of the API.

Solution: Check the key with dotnet user-secrets list and update the application.

The API answers 429.

Cause: More than 120 requests in one minute.

Solution: Wait one minute; the pending audit records are sent again automatically.

Error Number 208 in the API console.

Cause: A table does not exist in the database.

Solution: Run db/sacor_completo.sql on a new database.

«Network request failed» in the mobile app.

Cause: The phone cannot reach the address of the API.

Solution: Check EXPO_PUBLIC_API_URL, the firewall and that phone and server are on the same network.

An administrator cannot sign in.

Cause: Wrong user type for the application, or inactive employee.

Solution: Desktop needs AdministradorOperativo; mobile needs AdministradorTecnico.

HELP

Section 2

More problems

If the problem continues, collect the information below before asking for help.

«Ocurrió un error interno».

Cause: Unexpected error in the API.

Solution: Read the detail in the console of the API; the user only sees a generic message.

The website cannot send forms.

Cause: The address of the website is not an allowed origin.

Solution: Add it to Seguridad:OrigenesPermitidos and restart the API.

The installed APK does not connect.

Cause: Android blocks HTTP, or the APK has an old address.

Solution: Enable usesCleartextTraffic or use HTTPS, and build the APK again.

A weighing has no technician to choose.

Cause: The trip of that day has no technician assigned.

Solution: Assign a technician to the trip in the mobile app or in Salidas y camiones.

The WhatsApp bot does not answer.

Cause: The API console shows «firma invalida»: the webhook address does not match.

Solution: Write the same address in Twilio and in Twilio:UrlWebhook, and restart the API.

The bot stops answering after the tunnel restarts.

Cause: The quick tunnel created a new address.

Solution: Update the address in Sandbox settings and in Twilio:UrlWebhook.

Before asking for help

Write down the exact error message, the time, the user and what you were doing. Copy the lines of the API console from that moment, and check the table auditoria for the last changes.

HELP